Report - Team Deliverable 3 - The Pintos

Jovan Gelevski, John Burlando, Harry Carr, Mani Mir Mohammed, Ry Yebeyen, JP Gilliam

a. Requirements for Phase 1

- The system shall allow users to create an account using an email address and password.
- The system shall allow users to log in using the credentials specified during account creation.
- The system shall allow users to log out and terminate their active session.
- The system shall allow users to update account information such as email and password.
- The system shall allow users to permanently delete their account and associated data.
- The system shall allow users to add one or more vehicle profiles.
- The system shall allow users to manually enter vehicle details including make, model, year, mileage, fuel type, oil type, and recommended oil change interval.
- The system shall allow users to edit existing vehicle details.
- The system shall allow users to delete a vehicle profile.
- The system shall allow users to update the current mileage (odometer reading) of their vehicle at any time.
- The system shall allow users to log a maintenance event for a selected vehicle.
- The system shall allow users to select a service type (e.g., oil change, tire rotation, brake service, inspection).
- The system shall allow users to enter service date, odometer at time of service, and an optional description/notes section.
- The system shall allow users to edit an existing maintenance record.
- The system shall allow users to delete a maintenance record.
- The system shall display all maintenance records in chronological order.
- The system shall allow users to filter service history by service type.
- The system shall allow users to view detailed information for each maintenance entry.
- TuneUp provides automated reminder functionality to help users stay up to date with vehicle maintenance.
- The system shall allow users to configure maintenance reminders based on mileage intervals.
- The system shall allow users to configure maintenance reminders based on time intervals (e.g. every 6 months).
- The system shall display upcoming and overdue maintenance tasks on the dashboard.
- The system shall display all registered vehicles on the dashboard.
- The system shall display upcoming or overdue services.
- The system shall display recent maintenance entries.
- Pages shall load within 3 seconds under normal network conditions.
- Updates to local interface elements shall occur within 100 milliseconds.
    - Changed from previous requirement where timeframe was not concrete.
- Database updates shall complete within 3 seconds under normal network conditions.
    - Changed from previous requirement where timeframe was again not specified concretely.

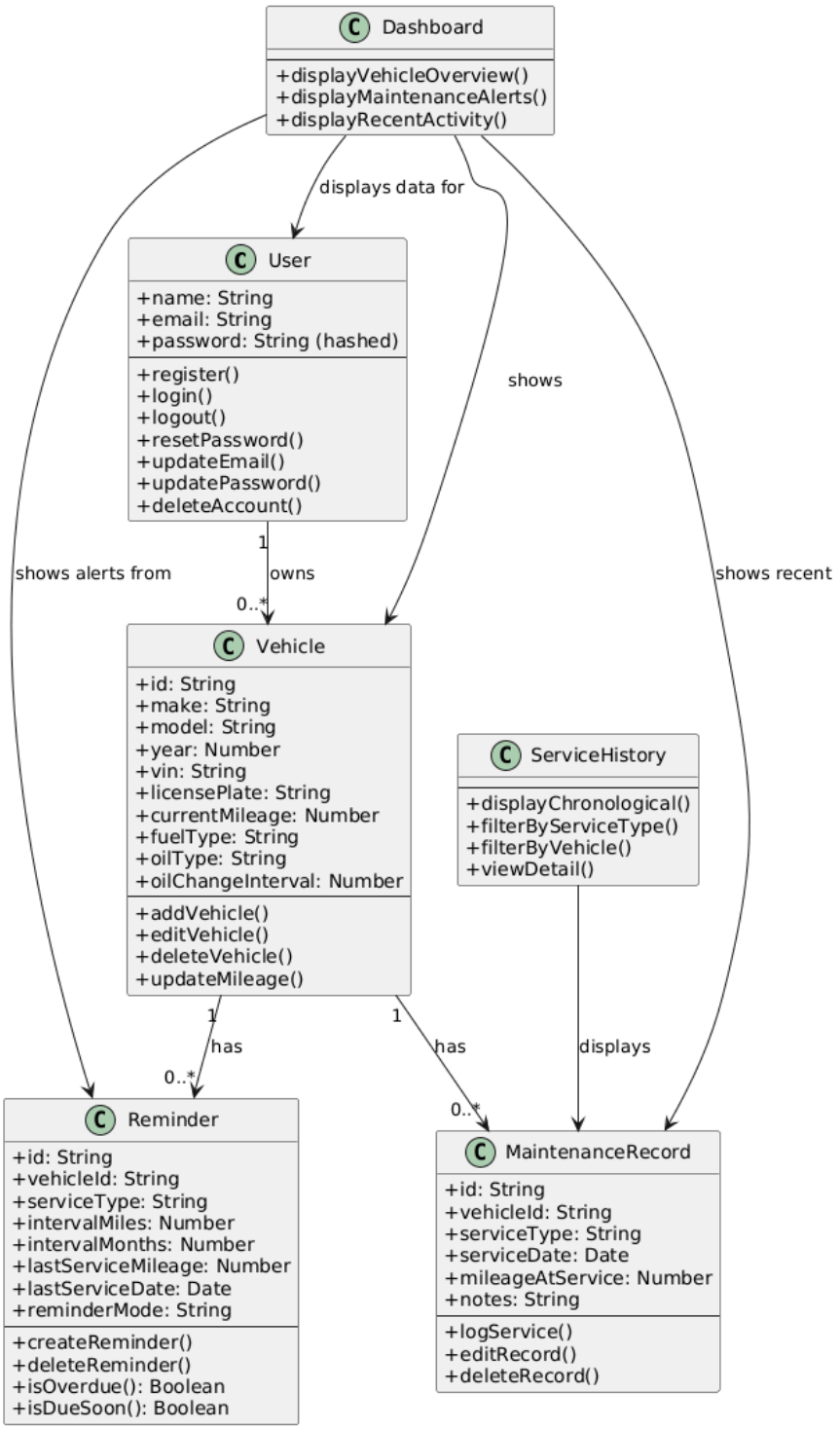
- Passwords shall be securely hashed with Bcrypt before being stored.
    - Changed from previous requirement where the hashing method was undefined.
- Users shall not be able to access or modify other users' data.
- All user input shall be validated to prevent common web vulnerabilities (e.g., SQL injection, cross-site scripting).
- The system shall function on desktop, tablet, and mobile devices.
- The interface shall allow new users to be able to create an account and log a maintenance record within 10 minutes without assistance.
- The system shall provide clear and descriptive error messages.
- The system shall maintain accurate and consistent storage of all user data.
- The system shall maintain at least 95% uptime during the project deployment period.
- The project shall use GitHub for version control.
- The system shall follow modular design principles to support future expansion.
- The system shall include setup and deployment documentation.
- The system shall handle user email and personal data in accordance with applicable privacy standards.

Requirements Rationale:

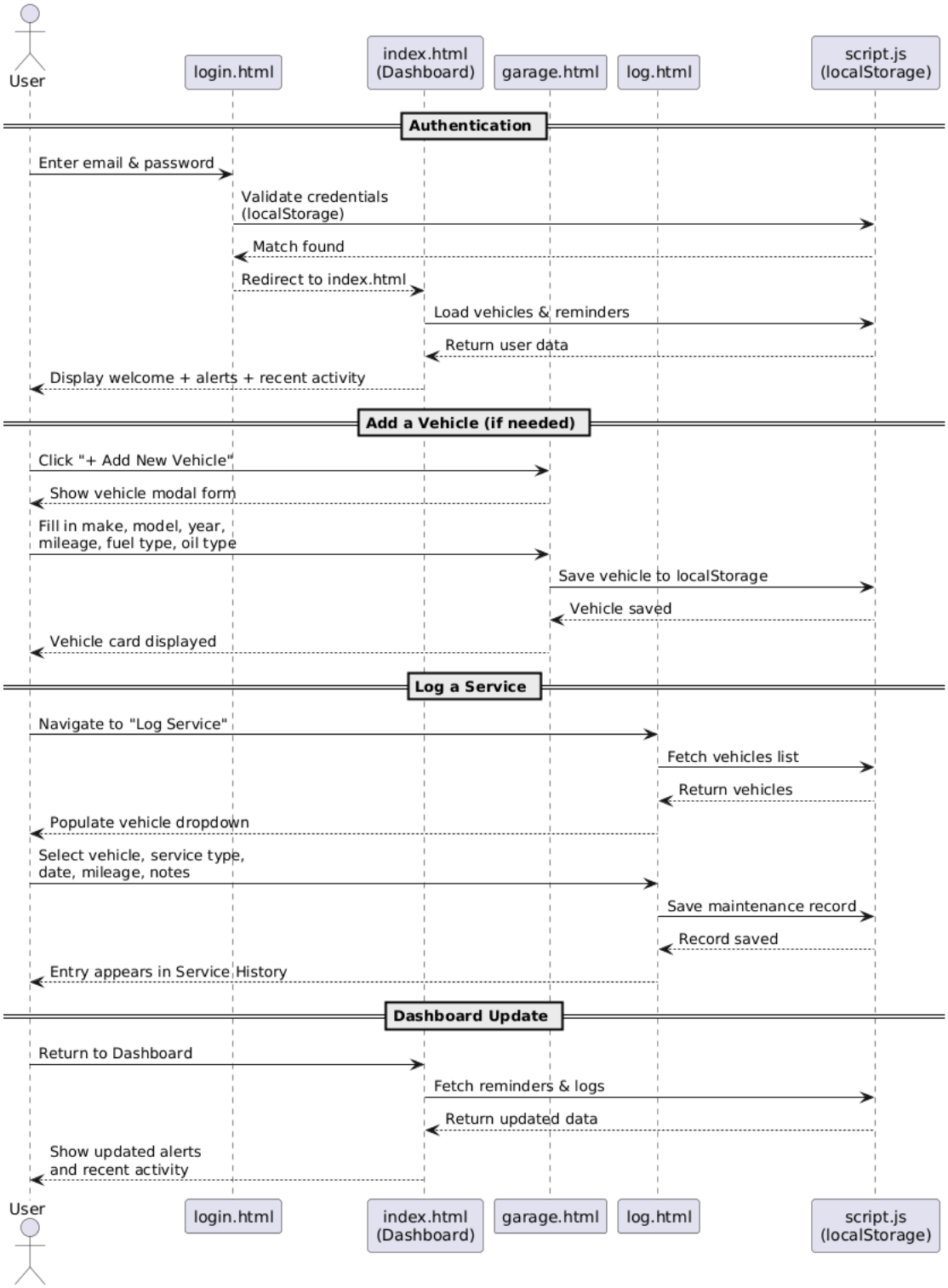
The requirements we set out to fulfill during phase one were mainly concerned with implementing core functionality such as the pages, the logic behind the pages, the data classes, data persistence, and other similar things. As phase one progressed and as it now comes to a close, we believe that we've ironed out a non-insignificant amount of requirements and that the project is getting close to the point where we can begin implementing the finer details. That being said, our scope from the start up until now has not changed significantly, and our requirements reflect this.

b. UML Diagrams

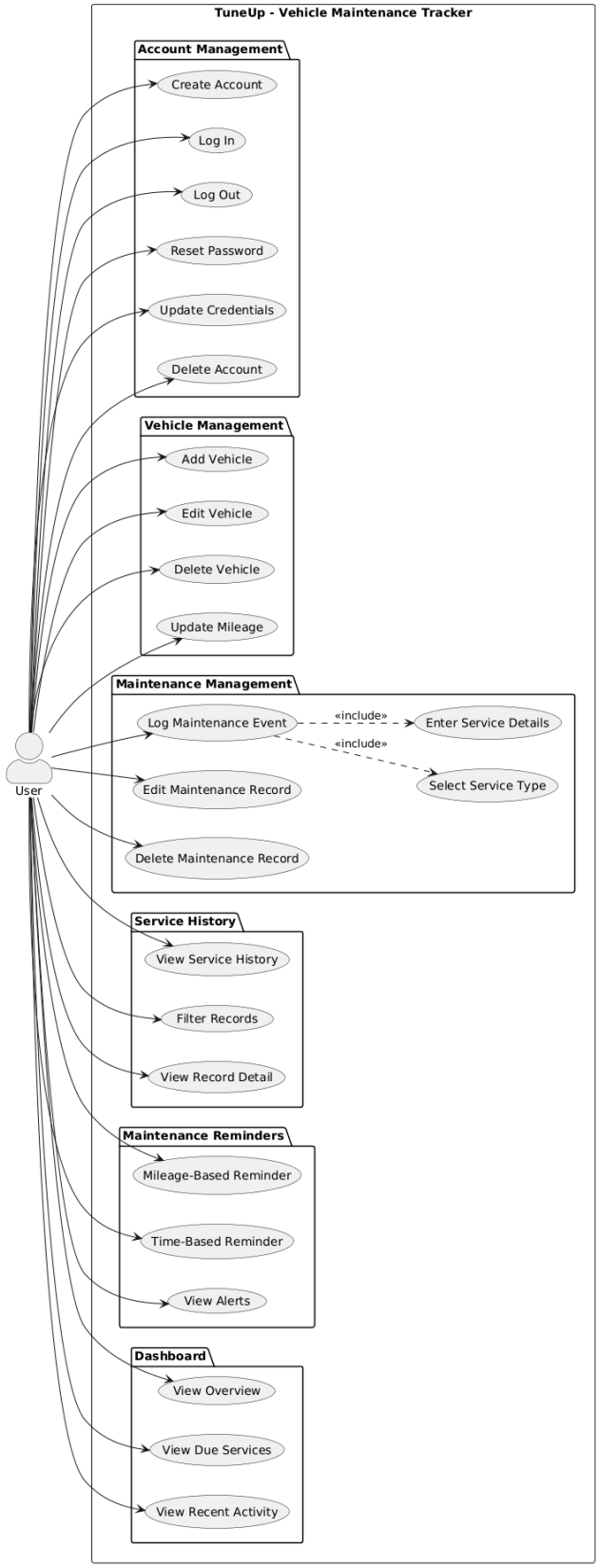
Class diagram -


Dashboard
+displayVehicleOverview()
+displayMaintenanceAlerts()
+displayRecentActivity()
displays data for
shows
shows alerts from
shows recent
User
+name: String
+email: String
+password: String (hashed)
+register()
+login()
+logout()
+resetPassword()
+updateEmail()
+updatePassword()
+deleteAccount()
1
owns
0..*
Vehicle
+id: String
+make: String
+model: String
+year: Number
+vin: String
+licensePlate: String
+currentMileage: Number
+fuelType: String
+oilType: String
+oilChangeInterval: Number
+addVehicle()
+editVehicle()
+deleteVehicle()
+updateMileage()
ServiceHistory
+displayChronological()
+filterByServiceType()
+filterByVehicle()
+viewDetail()
1
has
0..*
1
has
0..*
displays
Reminder
+id: String
+vehicleId: String
+serviceType: String
+intervalMiles: Number
+intervalMonths: Number
+lastServiceMileage: Number
+lastServiceDate: Date
+reminderMode: String
+createReminder()
+deleteReminder()
+isOverdue(): Boolean
+isDueSoon(): Boolean
MaintenanceRecord
+id: String
+vehicleId: String
+serviceType: String
+serviceDate: Date
+mileageAtService: Number
+notes: String
+logService()
+editRecord()
+deleteRecord()

Sequence diagram -



Use case diagram -



c. User Manual

TuneUp's set-up is as follows:
Software required: Visual Studio Code, Node.js and Node Packet Manager (npm), MySQL Server 8.0 or later
Clone the repository onto your local machine from the link provided in the comments or: https://github.com/jovanlevsky/ThePintos.git
Open the directory it was cloned into from an IDE of your choice (Visual Studio Code or Codium recommended).
In your file explorer, open the directory also and navigate to the "tuneup_backend" subfolder, and open two terminal tabs inside.
From within your IDE, create a file called ".env" within the "tuneup_backend" subfolder.
Configure MySQL on your machine, and set your root password for the service.
In .env, insert the following:

PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASS=<yourpasswordhere>
DB_NAME=tuneup
JWT_SECRET=randomretgehdfhdsfhsdfhsd
JWT_EXPIRES_IN=7d
CORS_ORIGIN=http://127.0.0.1:5500/

Replace the DB_PASS placeholder field with the root password you set for MySQL.
In your terminals, with one of them you'll type the following:
mysql -u root -p which will then prompt your root password.
Run the following: "SOURCE tuneupschema.sql;" this will create the database and default to it
In your other terminal, type the following: "npm install express mysql2 bcrypt jsonwebtoken dotenv express-validator cors"
Once those dependencies install, you can then use that same terminal tab and run "node server.js"
Now the local server will be running, upon which you can use something similar to the Live Server function in Visual Studio Code/Codium and host the website on your browser.
You are now able to operate the project.

d. Test cases

In order to run the tests, navigate to the backend directory like before and open a terminal and run the command: "npx jest tuneup.test.js" It will install the jest test package and run the tests.
Here's an overview of the coverage:
Adding a vehicle -
We have a test suite where a reasonable set of cases is tested when it comes to adding a vehicle, which lie in two main categories: local data being sent to API, and API data being sent to local. The tests cover whether:
When a vehicle is sent to local from the API, the id is converted to a string, the values map properly (license_plate -> plate, mileage, fueltype), which values are defaulted to when the api sends null for oil interval, vin and plate.

When a vehicle is sent from local storage to API, the year is converted to a number format from string, the mileage is converted from a string to a number, that the previous values such as vin and oil interval properly map to null when empty, and that the other fields map properly.
When the site is relying on localStorage, the proper values are returned when certain things are queried, such as getVehicles returning an empty array when nothing is stored, whether saving and getting vehicles works, and whether reminders properly render.
When ids are being generated, they're being parsed correctly and only unique ids are being generated.
When tokens are generated and issued, they're properly handled when they're null or expired.
When maintenance logs are sent from API to local and vice versa, the fields properly map and fields properly default to expected values if they are null or empty.
When reminders are sent from API to local and vice versa, again, the fields properly map to each other (interval_miles->intervalMiles) and default value mapping is handled properly.
When getVehicleAlerts() is called, is it properly returning an empty array when a car that doesn't exist is called, is it returning an empty array when it has no reminders, is it showing proper flags when mileage exceeds a certain threshold (10,000) or time goes beyond a certain point (6 months)?
And lastly, when the initials function is called for the profile page, is it properly outputting a one or two character initial even when it has more than two names inputted or lots of excess whitespace, is it parsing lowercase letters to uppercase when converting them to an initial?

e. Reflection

So far, we've managed to build the repository to the point where the majority of main functionality has been implemented fully or close to fully. Development in a requirements-driven way has been a unique experience for us and so far, it's going really well. One of the things that went very well this phase was overall coordination between the team. We were able to accomplish what we set out to do for this first phase because of that. Another thing is handling development incrementally went well for us. Not trying to implement the whole system at once, but starting small and scalable and building off of that gave us a platform to work with. Some things that could have been improved in this phase were time management, spending more time on building out the backend, and doing more frequent check-ins with each other.

f. Member Contribution Table

Member Name	Contribution Description	Overall Contribution (%)	Note (if applicable
Jovan	Worked on merging functionality and building my garage and log service pages and some unit tests for the backend. Wrote the user manual and did the UML diagrams.	16.7%	

JP	Worked on Dashboard logic and implementation, contributed to the code inspection documentation, helped develop test cases	16.7%	
John	Built framework site architecture, made contributions to the code inspection documentation.	16.7%	
Harry	Worked on the reminders tab	16.7%	
Ry	Worked on Login persistence and helped on inspection code	16.7%	
Mani			